Système d'Information de Gestion d'une Bibliothèque Universitaire (SIGBU)

Spécifications fonctionnelles, modélisation UML 2.5 multi-classes et implémentation en C++17

Référence	ARC-SIGBU-EX1-2026	**Cadre**	Travaux Pratiques — POO C++ & UML
Périmètre	Noyau métier (Catalogue, Usagers, Prêts)	**Outils**	UML 2.5 / C++17 Standard (STL)
Architecture	6 classes (Héritage, Agrégation, Association)	**Version**	2.0 — Modèle complet du cours

1. Contexte, Objectif de l'Exercice et Règles Métier

L'exercice consiste à concevoir et implémenter le noyau logiciel d'une bibliothèque universitaire (**SIGBU**) en appliquant les principes fondamentaux de la Programmation Orientée Objet : **encapsulation**, **héritage**, **polymorphisme** et gestion des **associations entre objets**. Le système doit garantir la cohérence du catalogue d'ouvrages, des usagers inscrits et du suivi des emprunts.

Règles métier strictes du cahier des charges

- **Unicité de détention :** Un exemplaire de `Livre` ne peut faire l'objet que d'un seul `Emprunt` actif à la fois (cardinalité `0..1`).
- **Multi-emprunt étudiant :** Un `Etudiant` peut détenir simultanément zéro à plusieurs ouvrages (cardinalité `0..*`).
- **Gestion du catalogue :** Le `Bibliothecaire` est habilité à ajouter ou retirer des ouvrages dans la `Bibliotheque`, tandis que l'`Etudiant` consulte le catalogue, emprunte et restitue des livres.
- **Intégrité d'état :** Lors d'un emprunt valide, l'attribut `disponible` du livre passe à `false` et un objet `Emprunt` est enregistré. Lors de la restitution, le livre redevient disponible (`true`) et l'emprunt est clôturé.

2. Analyse Fonctionnelle — Diagramme de Cas d'Utilisation

Conformément à l'énoncé, deux acteurs interagissent avec le système : l'**Étudiant** (usager principal) et le **Bibliothécaire** (acteur secondaire chargé de l'administration du fonds documentaire).

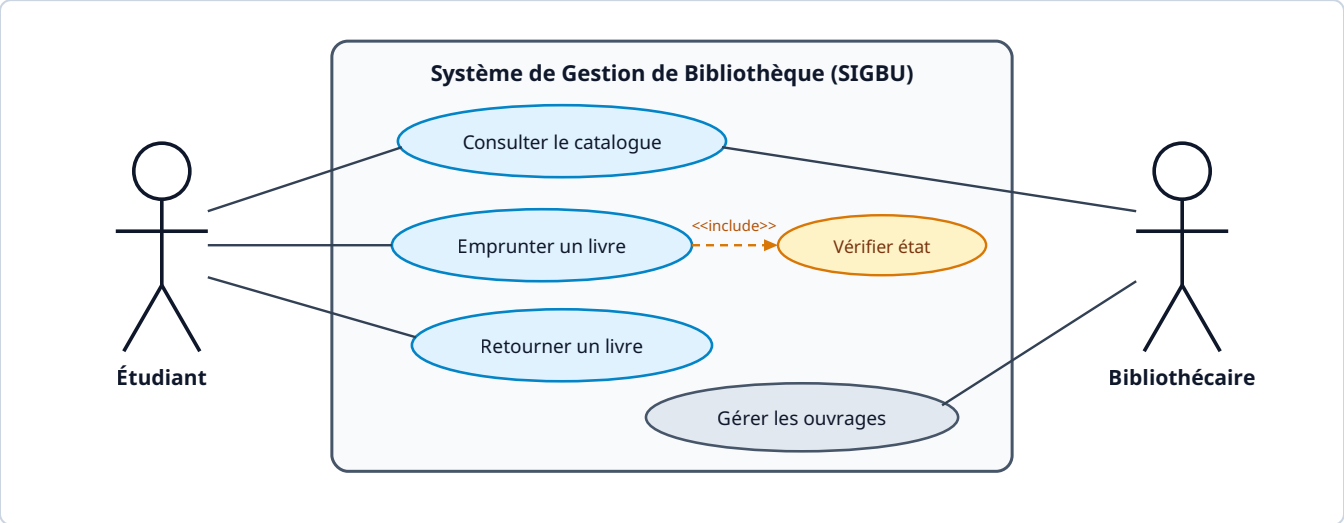


Figure 1 — Diagramme de cas d'utilisation UML du système SIGBU.

3. Conception Orientée Objet : Architecture Multi-Classes

Pour respecter fidèlement la structure vue en cours sans ajouter d'éléments hors sujet, l'architecture repose sur **six classes complémentaires** couvrant exactement les entités du domaine (usagers, personnel, catalogue, ouvrages et prêts) :

Classe & Rôle	Attributs (`protected` / `private`)	Méthodes principales (`public`)
Personne Classe mère (abstraite) factorisant les usagers	• `nom : string` (protected) • `prenom : string` (protected) • `email : string` (protected)	• `Personne(nom, prenom, email)` • `getNomComplet() const : string` • `afficherProfil() const : void` (virtuelle pure)
Etudiant Hérite de `Personne`	• `matricule : string` (Identifiant étudiant) • `livresEmpruntes : vector<Livre*>`	• `Etudiant(mat, nom, prenom, email)` • `emprunterLivre(Livre*) : bool` • `retournerLivre(Livre*) : bool` • `afficherProfil() const : void` (override)
Bibliothecaire Hérite de `Personne`	• `idEmploye : string` (Matricule agent)	• `Bibliothecaire(id, nom, prenom, email)` • `ajouterOuvrage(Bibliotheque&, Livre*) : void` • `afficherProfil() const : void` (override)
Livre Entité documentaire du catalogue	• `id : string`, `titre : string` • `auteur : string`, `anneePublication : int` • `disponible : bool`	• `Livre(id, titre, auteur, annee)` • `marquerEmprunte() : bool` • `marquerDisponible() : bool` • `estDisponible() const : bool`, `afficher() const`
Emprunt Classe d'association Prêt (Etudiant–Livre)	• `idEmprunt : int` • `etudiant : Etudiant*`, `livre : Livre*` • `dateEmprunt : string`, `enCours : bool`	• `Emprunt(id, Etudiant*, Livre*, date)` • `cloturer() : void` • `estEnCours() const : bool`, `afficher() const`
Bibliotheque Agrégateur central (Catalogue & Prêts)	• `nom : string` • `catalogue : vector<Livre*>` • `etudiants : vector<Etudiant*>` • `emprunts : vector<Emprunt>`	• `ajouterLivre(Livre*) : void` • `inscrireEtudiant(Etudiant*) : void` • `enregistrerEmprunt(Etudiant*, Livre*, date) : bool` • `enregistrerRetour(Etudiant*, Livre*) : bool` • `afficherCatalogue() const : void`

Justification des relations UML du cours

- **Généralisation / Héritage (`Personne` ← `Etudiant`, `Bibliothecaire`) :** Évite la duplication des attributs d'identité (`nom`, `prenom`, `email`) et illustre le polymorphisme via la méthode virtuelle pure `afficherProfil()`.

- **Agrégation (`Bibliotheque` ◇— `Livre` et `Etudiant`) :** La bibliothèque regroupe les ouvrages (`0..*`) et les étudiants inscrits (`0..*`) sans détruire leur existence autonome.
- **Classe d'association (`Emprunt`) :** Matérialise la transaction de prêt entre un `Etudiant` et un `Livre` en conservant la date d'emprunt et l'état du prêt (`enCours`).

4. Diagramme de Classes UML 2.5

Le diagramme ci-dessous synthétise l'ensemble des classes, leurs visibilités (`+` public, `#` protected, `-` private), les relations d'héritage, d'agrégation et les cardinalités associées.

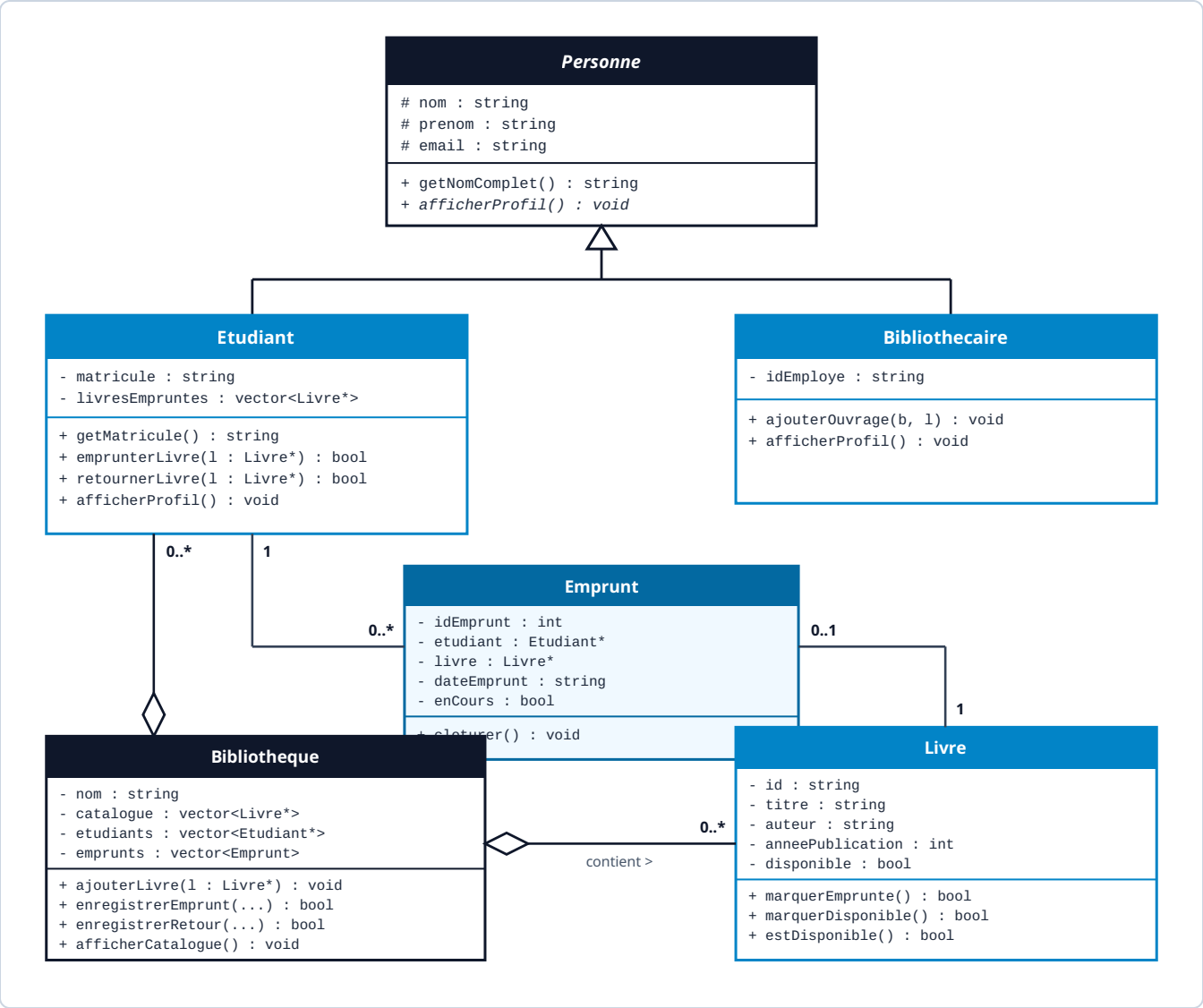


Figure 2 — Diagramme de classes UML complet (héritage, agrégation et classe d'association).

5. Implémentation Complète en C++17

Le code source ci-dessous implémente l'ensemble des six classes du modèle avec encapsulation stricte, destructeur virtuel sur la classe de base `Personne`, passage par référence constante et synchronisation cohérente entre `Etudiant`, `Livre`, `Emprunt` et `Bibliotheque`.

```
// ============================================================================
// FICHIER : sigbu_projet.cpp
// OBJET   : Noyau multi-classes de gestion de bibliothèque universitaire
// NORME   : C++17 (Compilation : g++ -std=c++17 -Wall -Wextra sigbu_projet.cpp)
// ============================================================================
#include <iostream>
#include <string>
#include <vector>
#include <algorithm>

using namespace std;

class Bibliotheque; // Déclaration anticipée

// ----------------------------------------------------------------------------
// 1. CLASSE MERE ABSTRAITE : Personne
// ----------------------------------------------------------------------------
class Personne {
protected:
    string nom;
    string prenom;
    string email;

public:
    Personne(const string& nom, const string& prenom, const string& email)
        : nom(nom), prenom(prenom), email(email) {}

    virtual ~Personne() = default;

    string getNomComplet() const { return prenom + " " + nom; }
    virtual void afficherProfil() const = 0; // Polymorphisme pur
};

// ----------------------------------------------------------------------------
// 2. CLASSE METIER : Livre
// ----------------------------------------------------------------------------
class Livre {
private:
    string id;
    string titre;
    string auteur;
    int anneePublication;
    bool disponible;

public:
    Livre(const string& id, const string& titre, const string& auteur, int annee)
        : id(id), titre(titre), auteur(auteur), anneePublication(annee), disponible(true) {}

    string getId() const { return id; }
    string getTitre() const { return titre; }
    bool estDisponible() const { return disponible; }

    bool marquerEmprunte() {
        if (!disponible) return false;
        disponible = false;
        return true;
    }

    bool marquerDisponible() {
```

```
        if (disponible) return false;
        disponible = true;
        return true;
    }

    void afficher() const {
        cout << "  [" << id << "] "" << titre << "" - " << auteur
             << " (" << anneePublication << ") | Etat : "
             << (disponible ? "DISPONIBLE" : "EMPRUNTE") << endl;
    }
};

// ---------------------------------------------------------------------------
// 3. CLASSE DERIVEE : Etudiant
// ---------------------------------------------------------------------------
class Etudiant : public Personne {
private:
    string matricule;
    vector<Livre*> livresEmpruntes;

public:
    Etudiant(const string& matricule, const string& nom,
             const string& prenom, const string& email)
        : Personne(nom, prenom, email), matricule(matricule) {}

    string getMatricule() const { return matricule; }

    bool emprunterLivre(Livre* livre) {
        if (livre == nullptr || !livre->marquerEmprunte()) return false;
        livresEmpruntes.push_back(livre);
        return true;
    }

    bool retournerLivre(Livre* livre) {
        if (livre == nullptr) return false;
        auto it = find(livresEmpruntes.begin(), livresEmpruntes.end(), livre);
        if (it != livresEmpruntes.end() && livre->marquerDisponible()) {
            livresEmpruntes.erase(it);
            return true;
        }
        return false;
    }

    void afficherProfil() const override {
        cout << "[ETUDIANT] " << getNomComplet() << " (" << matricule
             << ") | Livres en cours : " << livresEmpruntes.size() << endl;
    }
};
```

```cpp
// ----------------------------------------------------------------------------
// 4. CLASSE D'ASSOCIATION : Emprunt
// ----------------------------------------------------------------------------
class Emprunt {
private:
    int idEmprunt;
    Etudiant* etudiant;
    Livre* livre;
    string dateEmprunt;
    bool enCours;

public:
    Emprunt(int id, Etudiant* etu, Livre* liv, const string& date)
        : idEmprunt(id), etudiant(etu), livre(liv), dateEmprunt(date), enCours(true) {}

    Etudiant* getEtudiant() const { return etudiant; }
    Livre* getLivre() const { return livre; }
    bool estEnCours() const { return enCours; }
    void cloturer() { enCours = false; }

    void afficher() const {
        cout << "  Pret #" << idEmprunt << " | " << etudiant->getNomComplet()
             << " -> "" << livre->getTitre() << "" (" << dateEmprunt
             << ") | Statut : " << (enCours ? "ACTIF" : "CLOTURE") << endl;
    }
};

// ----------------------------------------------------------------------------
// 5. CLASSE AGREGATRICE : Bibliotheque
// ----------------------------------------------------------------------------
class Bibliotheque {
private:
    string nom;
    vector<Livre*> catalogue;
    vector<Etudiant*> etudiants;
    vector<Emprunt> emprunts;
    int compteurEmprunts = 1;

public:
    Bibliotheque(const string& nom) : nom(nom) {}

    void ajouterLivre(Livre* livre) {
        if (livre != nullptr) catalogue.push_back(livre);
    }

    void inscrireEtudiant(Etudiant* etu) {
        if (etu != nullptr) etudiants.push_back(etu);
    }

    bool enregistrerEmprunt(Etudiant* etu, Livre* livre, const string& date) {
        if (etu == nullptr || livre == nullptr) return false;
        if (etu->emprunterLivre(livre)) {
            emprunts.emplace_back(compteurEmprunts++, etu, livre, date);
            cout << "[OK] Emprunt valide : " << etu->getNomComplet()
                 << " emprunte "" << livre->getTitre() << """ << endl;
            return true;
        }
        cout << "[REFUS] Ouvrage indisponible : "" << livre->getTitre() << """ << endl;
        return false;
    }

    bool enregistrerRetour(Etudiant* etu, Livre* livre) {
        if (etu == nullptr || livre == nullptr) return false;
        if (etu->retournerLivre(livre)) {
            for (auto& emp : emprunts) {
                if (emp.estEnCours() && emp.getEtudiant() == etu && emp.getLivre() == livre) {
                    emp.cloturer();
```

```
                    break;
                }
            }
            cout << "[OK] Restitution enregistree : "" << livre->getTitre() << """ << endl;
            return true;
        }
        cout << "[ERREUR] Ce livre n'est pas detenu par " << etu->getNomComplet() << endl;
        return false;
    }

    void afficherCatalogue() const {
        cout << "--- Catalogue de " << nom << " ---" << endl;
        for (const auto* l : catalogue) l->afficher();
    }

    void afficherEmprunts() const {
        cout << "--- Registre des Emprunts ---" << endl;
        for (const auto& emp : emprunts) emp.afficher();
    }
};

// ------------------------------------------------------------------------
// 6. CLASSE DERIVEE : Bibliothecaire
// ------------------------------------------------------------------------
class Bibliothecaire : public Personne {
private:
    string idEmploye;

public:
    Bibliothecaire(const string& idEmp, const string& nom,
                   const string& prenom, const string& email)
        : Personne(nom, prenom, email), idEmploye(idEmp) {}

    void ajouterOuvrage(Bibliotheque& bib, Livre* livre) const {
        bib.ajouterLivre(livre);
        cout << "[ADMIN] " << getNomComplet() << " a ajoute ""
             << livre->getTitre() << "" au catalogue." << endl;
    }

    void afficherProfil() const override {
        cout << "[BIBLIOTHECAIRE] " << getNomComplet() << " (Agent " << idEmploye << ")" << endl;
    }
};
```

```cpp
// ----------------------------------------------------------------------------
// PROGRAMME PRINCIPAL : Scénario de validation complet
// ----------------------------------------------------------------------------
int main() {
    Bibliotheque sigbu("Bibliotheque Universitaire Centrale");

    // Creation des acteurs (heritage Personne) et des ouvrages
    Bibliothecaire bib1("EMP-01", "Diallo", "Moussa", "m.diallo@univ.edu");
    Etudiant etu1("2026-0042", "Mensah", "Afi", "afi.mensah@univ.edu");
    Etudiant etu2("2026-0118", "Kouassi", "Jean", "jean.kouassi@univ.edu");

    Livre l1("INF-101", "Architecture Logicielle et UML", "Grady Booch", 2020);
    Livre l2("INF-204", "Programmation Systeme en C++", "Bjarne Stroustrup", 2022);

    // Administration du catalogue par le bibliothecaire
    bib1.ajouterOuvrage(sigbu, &l1);
    bib1.ajouterOuvrage(sigbu, &l2);
    sigbu.inscrireEtudiant(&etu1);
    sigbu.inscrireEtudiant(&etu2);

    // Demonstration du polymorphisme sur Personne*
    vector<Personne*> acteurs = { &bib1, &etu1, &etu2 };
    for (const auto* p : acteurs) p->afficherProfil();

    // Transactions d'emprunt et de restitution
    sigbu.enregistrerEmprunt(&etu1, &l1, "2026-11-12");
    sigbu.enregistrerEmprunt(&etu1, &l2, "2026-11-12");
    sigbu.enregistrerEmprunt(&etu2, &l1, "2026-11-12"); // Refus attendu

    sigbu.enregistrerRetour(&etu1, &l1);
    sigbu.enregistrerEmprunt(&etu2, &l1, "2026-11-12"); // Reattribution valide

    sigbu.afficherCatalogue();
    sigbu.afficherEmprunts();

    return 0;
}
```

6. Plan de Tests et Validation des Exigences

Le scénario d'exécution du programme principal vérifie le bon fonctionnement de chaque classe et le respect des invariants du domaine :

Cas de Test	Action exécutée	Résultat attendu et vérifié
TC-01	`bib1.ajouterOuvrage(sigbu, &l1)`	Le `Bibliothecaire` alimente le catalogue de la `Bibliotheque` ; le livre est initialisé à `disponible = true`.
TC-02	Appel de `p->afficherProfil()` via `Personne*`	Résolution dynamique correcte entre `Bibliothecaire` et `Etudiant` (validation de l'héritage et du polymorphisme).
TC-03	`sigbu.enregistrerEmprunt(&etu1, &l1, ...)` puis `&l2`	Création de deux instances `Emprunt` actives ; `etu1` détient 2 ouvrages simultanés (cardinalité `0..*`).
TC-04	`sigbu.enregistrerEmprunt(&etu2, &l1, ...)` alors que `l1` est prêté	Rejet immédiat (`[REFUS]`) sans création d'emprunt parasite (unicité de détention `0..1` garantie).

Cas de Test	Action exécutée	Résultat attendu et vérifié
TC-05	`sigbu.enregistrerRetour(&etu1, &l1)` suivi d'un prêt à `etu2`	Le premier `Emprunt` passe à `CLOTURE`, `l1` redevient disponible, puis un nouvel `Emprunt` actif est créé pour `etu2`.